

GREYNOC

Coding Bot Setup Guide

Public-facing guidance for using Markdown instructions, validation scripts, and repeatable workflows to improve AI-assisted software development.

PUBLIC / APPROVED FOR EXTERNAL RELEASE
GN-AI-CODEBOT-001 | v1.1

Website: greynoc.com
Contact: customerservice@greynoc.com
Distribution: Public / Approved external release
Copyright: © 2026 GreyNOC. All rights reserved.

Public Release Position

This guide provides defensive, educational guidance for setting up coding bots in software repositories. It is intended to help developers, small businesses, IT teams, cybersecurity teams, and technical operators create safer and more repeatable AI-assisted development workflows.

This guide does not guarantee secure code, replace human review, replace secure software development practices, or authorize automated changes to production systems without approval. Coding bots should be treated as development assistants. Human owners remain responsible for requirements, security review, testing, deployment decisions, and final approval.

Template visual alignment
This v1.1 release applies the Google Drive public document template language, navy/red palette, release-safe classification wording, scan-friendly tables, and public release checklist structure.

CORE PRINCIPLE: Markdown gives the bot context. Scripts prove whether the work passes checks. A repeatable workflow keeps the bot from guessing. Human review decides what gets merged.

00 Document Control

Visible ownership, scope, review state, and release boundaries are required before public GreyNOC publication.

Field	Value	Notes
Title	Coding Bot Setup Guide	Public-facing AI-assisted development guide.
Document ID	GN-AI-CODEBOT-001	GreyNOC AI / coding workflow publication tracker.
Classification	PUBLIC / APPROVED FOR EXTERNAL RELEASE	Approved external release classification.
Version	v1.1	QA/QC release bump; document-control metadata, header/footer release markers, publication decision, accessibility/style checks, and fixed-layout render review complete.

GREYNOC CYBERSECURITY OPERATIONS

Coding Bot Setup Guide
PUBLIC | GN-AI-CODEBOT-001

Field	Value	Notes
Effective Date	2026-06-15	Release date for the bumped public release package.
Owner	GreyNOC Communications / Cybersecurity Operations	Single accountable owner for source, PDF, references, approval notes, and publication record.
Approvers	Owner, Security, Legal, Communications	Approval record retained by document owner.
Template Basis	GreyNOC Public Document Template GN-TPL-PUB-001 v1.0	Aligned to the approved public template structure, including document control, release rules, standard content layout, release checklist, archival requirement, and publication notes.

Version History

Version	Date	Author	Summary of Change
v0.1	2026-06-14	GreyNOC	Initial public-facing draft aligned to GreyNOC release structure.
v0.2	2026-06-14	GreyNOC	Added Google Drive public-template visual styling, refined layout, and strengthened release-safe formatting.
v0.3	2026-06-14	GreyNOC	Expanded operator knowledge sections with runbooks, triage methods, failure modes, governance controls, and metrics.
v1.0	2026-06-14	GreyNOC	Final QA/QC release: template alignment, versioning, spacing, page rendering, footer/header consistency, and publication metadata checked.
v1.1	2026-06-15	GreyNOC	QA/QC release bump: refreshed document-control metadata, header/footer release markers, final publication decision, accessibility/style checks, and fixed-layout render review.

Release Safety Review

Area	Release Finding	Decision
Security	The guide focuses on safe development workflow, validation scripts, secret prevention, human approval, and controlled repository changes.	Pass - final QA/QC complete
Privacy	The guide does not request personal data, customer data, credentials, private repositories, or employee monitoring data.	Pass - final QA/QC complete
Legal / Operational	The guide does not authorize automated production deployment, destructive changes, or bypassing approval processes.	Control retained - no automated production authorization
Claims	The guide avoids promising that coding bots produce secure, correct, or production-ready code without testing and human review.	Pass - no guarantees or unsupported claims

Area	Release Finding	Decision
Communications	Tone is practical, restrained, public-facing, and focused on repeatable development hygiene.	Pass - final QA/QC complete
AI Quality	The guide gives specific files, scripts, checklists, and acceptance criteria instead of generic AI advice.	Pass - final QA/QC complete

PUBLIC RELEASE RULE: This guide may be used to improve AI-assisted development discipline. It must not be presented as a guarantee of secure code, automated compliance, production readiness, or vulnerability-free software.

01 Contents

Section	Title	Purpose
01	Executive Summary	Explain why structured coding bot setup matters.
02	Scope and Audience	Define who should use the guide and what is out of scope.
03	Operating Model	Show how Markdown, scripts, and workflow combine.
04	Repository Structure	Provide a practical starter layout.
05	Markdown Instruction Files	Define files that guide bot behavior.
06	Validation Scripts	Provide setup, test, lint, security, and build commands.
07	Task Template	Give humans a better way to assign coding work.
08	Coding Bot Algorithm	Define the step-by-step bot workflow.
09	Human Review Controls	Explain review, approval, and merge gates.
10	CI / Pull Request Controls	Align local validation with automated checks.
11	Security and Privacy Guardrails	Reduce common bot-generated code risks.
12	Rollout Plan	Provide phased adoption.
13	Assumptions, Limitations, and Dependencies	State what this guide can and cannot prove.
14	Public Release Checklist and References	Prepare the document for external release.
15	Publication Metadata	Record final publication status.
A	Operator Knowledge Expansion	Advanced operator guidance for scoping, validation, triage, incident response, metrics, and prompt patterns.

01 Executive Summary

Coding bots perform better when a repository gives them the same things a capable developer needs: project context, architecture rules, coding standards, security boundaries, validation commands, and a clear definition of done.

GreyNOC recommends treating coding bot setup as an operational workflow, not as a one-time prompt. A strong setup uses Markdown files to guide behavior, scripts to validate results, and human review to decide what gets merged.

		Recommended decision Adopt the starter repository controls first, then expand to CI gates and pull request review. Do not allow bots to bypass existing owner approval.	
Priority	Recommended Action	Expected Outcome	Effort

GREYNOC CYBERSECURITY OPERATIONS

Coding Bot Setup Guide
PUBLIC | GN-AI-CODEBOT-001

Priority	Recommended Action	Expected Outcome	Effort
Now	Add AGENTS.md, .bot/PROJECT.md, and scripts/validate.sh.	Gives the bot basic context and one validation command.	Low
Next 30 Days	Add coding standards, security rules, task templates, and review checklist.	Reduces vague prompts, unsafe changes, and review confusion.	Medium
Next 90 Days	Align validation scripts with CI and pull request checks.	Makes local bot validation match merge requirements.	Medium
Ongoing	Review bot failures and update instructions/scripts.	Improves bot output over time.	Medium

Area	Status	Summary
Development Readiness	Moderate	Coding bots can help immediately, but only when scoped and validated.
Security Exposure	Moderate	Main risks are secrets, auth changes, unsafe dependencies, and unreviewed code.
Recommended Priority	P1	Establish repository instructions and validation scripts before giving bots broad edit access.

02 Scope and Audience

Item	Public-Facing Scope
Audience	Developers, cybersecurity teams, small businesses, IT teams, MSPs, technical operators, and leaders using AI-assisted coding tools.
In Scope	Repository instructions, Markdown bot files, validation scripts, task templates, review checklists, CI alignment, security guardrails, and controlled development workflows.
Out of Scope	Fully autonomous production deployment, bypassing human review, exploit development, credential theft, evasion, malware creation, secret handling, and customer-specific architecture.
Use Boundary	Use this as public AI-assisted development guidance. Adapt commands to the language, framework, and risk level of each project.
Approval Boundary	Do not merge, deploy, or publish bot-generated code without the normal owner review, security review, test evidence, and release approval required by the organization.

03 Operating Model

A coding bot needs structured inputs and controlled feedback. The operating model below gives the bot context, gives the team proof, and keeps humans accountable for release decisions.

Layer	Purpose	Example
Markdown Instructions	Tell the bot what matters.	AGENTS.md, .bot/SECURITY.md
Scripts	Prove whether the work passes checks.	scripts/validate.sh
Task Template	Tell the bot what to do and what not to change.	.bot/current-task.md
CI Workflow	Re-run validation outside the bot session.	GitHub Actions, GitLab CI, local pipeline
Human Review	Decide whether the change is acceptable.	Pull request review

CORE RULE: Do not measure a coding bot by how fast it writes code. Measure it by whether it produces small, understandable, validated changes that humans can safely review.

04 Repository Structure

GreyNOC recommends placing bot instructions and validation scripts directly in the repository so they are version-controlled and reviewable.

```
your-project/
├── .github/
│   ├── copilot-instructions.md
│   ├── pull_request_template.md
│   └── workflows/
│       └── ci.yml
├── .bot/
│   ├── PROJECT.md
│   ├── ARCHITECTURE.md
│   ├── CODING_STANDARDS.md
│   ├── SECURITY.md
│   ├── TASK_TEMPLATE.md
│   ├── REVIEW_CHECKLIST.md
│   ├── DONE_DEFINITION.md
│   └── examples/
│       ├── good-task.md
│       └── bad-task.md
├── scripts/
│   ├── setup.sh
│   ├── format.sh
│   ├── lint.sh
│   ├── test.sh
│   ├── security-check.sh
│   ├── build.sh
│   └── validate.sh
├── AGENTS.md
├── CLAUDE.md
├── README.md
└── src/
```

Minimum Starter Set

```
AGENTS.md
.bot/PROJECT.md
.bot/CODING_STANDARDS.md
.bot/SECURITY.md
.bot/DONE_DEFINITION.md
scripts/setup.sh
scripts/test.sh
scripts/validate.sh
```

05 Markdown Instruction Files

Markdown instruction files are the bot operating manual. They should be specific, short enough to follow, and reviewed like code.

05.1 AGENTS.md

```
# AGENTS.md

## Project Rule
This repository uses GreyNOC coding bot controls. Read this file before making changes.
```

```
## Required Reading
Before editing code, read:
- `.bot/PROJECT.md`
- `.bot/ARCHITECTURE.md` if the task affects design
- `.bot/CODING_STANDARDS.md`
- `.bot/SECURITY.md`
- `.bot/DONE_DEFINITION.md`
- `.bot/current-task.md` if present

## Required Behavior
Before editing:
1. Understand the task.
2. Identify files likely affected.
3. Create a short implementation plan.
4. Ask for clarification if scope is unsafe or unclear.

While editing:
1. Make the smallest complete change.
2. Stay within task scope.
3. Preserve existing architecture.
4. Do not reformat unrelated files.
5. Do not disable tests, linting, type checks, or security checks.
6. Do not add dependencies unless justified.

After editing:
1. Run formatting.
2. Run linting.
3. Run tests.
4. Run security checks.
5. Run the full validation command.
6. Summarize results clearly.

## Validation Command
Run: `scripts/validate.sh`

## Human Approval Required
Ask for human approval before deleting files, running destructive commands, changing authentication or authorization, adding dependencies, modifying database migrations, changing deployment settings, touching production infrastructure, or disabling tests/security checks.
```

05.2 .bot/PROJECT.md

```
# Project Overview

## Name
[Project Name]
```

```
## Purpose
[Plain-language explanation of what this project does.]

## Users
[Who uses it.]

## Main Capabilities
- [Capability 1]
- [Capability 2]
- [Capability 3]

## Important Constraints
- Security matters more than speed.
- Code must be easy to review.
- The bot should avoid unnecessary rewrites.
- Every meaningful change should be validated.
- Human review is required before merging.
```

05.3 .bot/ARCHITECTURE.md

```
# Architecture

## System Pattern
This project uses the following structure:
1. Interface layer
2. Application layer
3. Domain layer
4. Infrastructure layer

## Rules
- Keep business logic out of UI and controller code.
- Keep external service calls in infrastructure modules.
- Keep domain logic testable without network access.
- Avoid circular imports.
- Avoid large files with mixed responsibilities.
- Follow existing patterns before introducing new ones.
```

05.4 .bot/CODING_STANDARDS.md

```
# Coding Standards

## General
- Write simple, readable code.
- Use descriptive names.
- Keep functions focused.
- Prefer explicit error handling.
- Avoid hidden side effects.
- Prefer boring, maintainable solutions.
```

```
## Testing
Every meaningful behavior change should include unit, integration, regression, security, or manual verification coverage.

## Documentation
Update documentation when commands, APIs, configuration, environment variables, or security-sensitive behavior changes.
```

05.5 .bot/SECURITY.md

```
# Security Rules

## Never Do These
- Do not commit secrets.
- Do not hard-code credentials.
- Do not disable authentication.
- Do not bypass authorization checks.
- Do not remove audit logs.
- Do not expose private data.
- Do not add dependencies without justification.
- Do not ignore failing security checks.

## Required Practices
- Validate inputs.
- Use parameterized database queries.
- Avoid unsafe shell execution.
- Avoid path traversal.
- Avoid unsafe deserialization.
- Avoid logging tokens, passwords, or personal data.
- Preserve least privilege.
```

05.6 .bot/DONE_DEFINITION.md

```
# Definition of Done

A task is done when:
- Requested behavior is implemented.
- Change is small and reviewable.
- Existing behavior is preserved unless the task says otherwise.
- Tests are added or updated when appropriate.
- Documentation is updated when appropriate.
- Formatting passes.
- Linting passes.
- Tests pass.
- Security checks pass.
- Build passes.
```

The final bot response must include summary, files changed, commands run, test results, security results, known limitations, and suggested human review focus.

06 Validation Scripts

Validation scripts give humans and bots stable commands. They should be easy to run locally, fail clearly, match CI, and avoid destructive actions.

Rule	Reason
Scripts should be easy to run locally.	Bots and humans need the same workflow.
Scripts should fail clearly.	Failures should be fixable without guessing.
Scripts should not hide errors.	False success creates unsafe confidence.
Scripts should match CI.	Local validation should reflect merge checks.
Scripts should avoid destructive actions.	Validation should not damage data or environments.

06.1 scripts/setup.sh

```
#!/usr/bin/env bash
set -euo pipefail

echo "Setting up project..."

if [ -f package.json ]; then
  npm install
fi

if [ -f pyproject.toml ]; then
  python3 -m pip install --upgrade pip
  python3 -m pip install -e ".[dev]" || true
fi

if [ -f requirements.txt ]; then
  python3 -m pip install -r requirements.txt
fi

if [ -f go.mod ]; then
  go mod download
fi

echo "Setup complete."
```

06.2 scripts/format.sh

```
#!/usr/bin/env bash
set -euo pipefail

echo "Formatting project..."
```

```
if [ -f package.json ]; then
  npm run format --if-present
fi

if command -v ruff >/dev/null 2>&1; then
  ruff format .
fi

if [ -f go.mod ]; then
  gofmt -w .
fi

echo "Formatting complete."
```

06.3 scripts/lint.sh

```
#!/usr/bin/env bash
set -euo pipefail

echo "Running lint checks..."

if [ -f package.json ]; then
  npm run lint --if-present
fi

if command -v ruff >/dev/null 2>&1; then
  ruff check .
fi

if [ -f go.mod ]; then
  go vet ./...
fi

echo "Lint checks complete."
```

06.4 scripts/test.sh

```
#!/usr/bin/env bash
set -euo pipefail

echo "Running tests..."

if [ -f package.json ]; then
  npm test --if-present
fi
```

```
if command -v pytest >/dev/null 2>&1; then
    pytest
fi

if [ -f go.mod ]; then
    go test ./...
fi

echo "Tests complete."
```

06.5 scripts/security-check.sh

```
#!/usr/bin/env bash
set -euo pipefail

echo "Running security checks..."

if command -v git >/dev/null 2>&1; then
    if git grep -n -I -E "(api[_-]?key|secret|password|private[_-]?key|token)" -- . \
        '!.package-lock.json' \
        '!.pnpm-lock.yaml' \
        '!.yarn.lock' \
        '!.AGENTS.md' \
        '!.bot/SECURITY.md'; then
        echo "Potential secret-like string found. Review required."
        exit 1
    fi
fi

if [ -f package.json ]; then
    npm audit --audit-level=high || true
fi

echo "Security checks complete."
```

06.6 scripts/build.sh

```
#!/usr/bin/env bash
set -euo pipefail

echo "Building project..."

if [ -f package.json ]; then
    npm run build --if-present
```

```
fi

if [ -f pyproject.toml ]; then
    python3 -m compileall .
fi

if [ -f go.mod ]; then
    go build ./...
fi

echo "Build complete."
```

06.7 scripts/validate.sh

```
#!/usr/bin/env bash
set -euo pipefail

echo "Starting full validation..."

scripts/format.sh
scripts/lint.sh
scripts/test.sh
scripts/security-check.sh
scripts/build.sh

echo "Validation complete. All required checks passed."
```

Make scripts executable

```
chmod +x scripts/*.sh
```

07 Task Template

Vague tasks produce vague code. Use a task template so the bot knows what to do and what not to change.

```
# Coding Bot Task

## Task Title
[Short title]

## Goal
[Describe the desired outcome in plain language.]

## Background
[Explain why the change is needed.]

## Scope
The bot may change:
- [Allowed file or folder]
```

The bot must not change:

- [Restricted file or folder]
- [Restricted behavior]

Requirements

- [Requirement 1]
- [Requirement 2]
- [Requirement 3]

Acceptance Criteria

The task is complete when:

- [Criterion 1]
- [Criterion 2]
- [Criterion 3]

Validation Commands

Run: scripts/validate.sh

Security Considerations

[Authentication, authorization, input validation, logging, dependency, or privacy concerns.]

Notes

[Examples, edge cases, or related links.]

Good Task Example

Coding Bot Task

Task Title

Add email validation to the registration form.

Goal

Prevent invalid email addresses from being submitted during registration.

Scope

The bot may change:

- src/features/register/
- src/lib/validation/
- Related tests

The bot must not change:

- Login behavior
- Password reset behavior
- Database schema
- Email provider configuration

```
## Requirements
- Reject empty email input.
- Reject malformed email input.
- Show a user-friendly error message.
- Do not log submitted email addresses.
- Add or update tests.

## Acceptance Criteria
- Empty email input shows an error.
- Invalid email input shows an error.
- Valid email input can be submitted.
- Existing registration tests still pass.
- New validation tests pass.

## Validation
Run: scripts/validate.sh
```

08 Coding Bot Algorithm

The bot should follow a repeatable process instead of jumping directly into code.

Input:

- Human task
- Repository files
- Markdown instructions
- Validation scripts

Output:

- Small, reviewable code change
- Test results
- Security results
- Summary

Algorithm:

1. Read repository instructions.
2. Read the task.
3. Inspect relevant files.
4. Identify constraints and risks.
5. Create a short implementation plan.
6. Make the smallest complete code change.
7. Add or update tests.
8. Run formatting.
9. Run linting.
10. Run tests.
11. Run security checks.

12. Run build.
13. Run full validation.
14. If checks fail, fix and repeat.
15. Summarize changes and limitations.

Practical Prompt

You are working in this repository.

First, read:

- AGENTS.md
- .bot/PROJECT.md
- .bot/CODING_STANDARDS.md
- .bot/SECURITY.md
- .bot/DONE_DEFINITION.md
- .bot/current-task.md

Follow these rules:

- Make the smallest complete change.
- Stay within task scope.
- Do not rewrite unrelated files.
- Do not add dependencies unless justified.
- Do not weaken security controls.
- Add or update tests when behavior changes.
- Update documentation when commands, configuration, or public behavior changes.

Before final response, run:

scripts/validate.sh

Final response must include:

1. Summary of changes
2. Files changed
3. Tests run
4. Security checks run
5. Build result
6. Known limitations
7. Suggested human review focus

09 Human Review Controls

Human review is the control that prevents bot speed from becoming operational risk.

```
# Bot-Generated Code Review Checklist
```

```
## Task Fit
```

- Does the change solve the requested task?
- Did the bot stay within scope?
- Did the bot avoid unrelated rewrites?
- Are acceptance criteria satisfied?

Code Quality

- Is the code simple?
- Is the code readable?
- Are names clear?
- Are functions focused?
- Does the code follow existing patterns?

Testing

- Are tests included or updated?
- Do tests cover expected behavior?
- Do tests cover failure cases?
- Did validation pass?

Security

- Are inputs validated?
- Are outputs safe?
- Are secrets protected?
- Are permissions preserved?
- Are logs safe?
- Were new dependencies reviewed?

Maintainability

- Is documentation updated?
- Are config changes explained?
- Is rollback realistic?

- Require human approval before merge to protected branches.
- Require human approval before production deployment.
- Require human approval before dependency changes, migrations, auth changes, infrastructure changes, or customer-facing behavior changes.

10 CI / Pull Request Controls

Local validation and CI validation should match as closely as possible.

GitHub Actions Example

```
name: CI
```

```
on:
```

```
  pull_request:
```

```
push:
  branches:
    - main

jobs:
  validate:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Node if needed
        if: hashFiles('package.json') != ''
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Set up Python if needed
        if: hashFiles('pyproject.toml') != '' || hashFiles('requirements.txt') != ''
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Set up Go if needed
        if: hashFiles('go.mod') != ''
        uses: actions/setup-go@v5
        with:
          go-version: '1.22'

      - name: Make scripts executable
        run: chmod +x scripts/*.sh

      - name: Setup
        run: scripts/setup.sh

      - name: Validate
        run: scripts/validate.sh
```

Pull Request Template

```
# Pull Request

## Summary
```

[Explain what changed.]

Task

[Link or paste the task the bot was given.]

Files Changed

[List major files changed.]

Validation

Commands run:

scripts/validate.sh

Results:

- Formatting:
- Linting:
- Tests:
- Security:
- Build:

Security Review

- [] No secrets added
- [] Authentication preserved
- [] Authorization preserved
- [] Input validation considered
- [] Sensitive logging avoided
- [] Dependencies reviewed

Human Review Checklist

- [] Change matches task
- [] Scope is appropriate
- [] Code is readable
- [] Tests are sufficient
- [] Documentation is updated if needed

11 Security and Privacy Guardrails

Coding bots can produce useful code, but they can also introduce risk. The safest pattern is to combine clear rules, validation scripts, and review gates.

Risk	Example	Control
Secret leakage	Bot commits an API key.	Secret scanning and review.
Auth bypass	Bot disables login to pass a test.	Security rules and protected tests.
Unsafe dependency	Bot adds a package unnecessarily.	Dependency justification rule.
Broad rewrite	Bot changes unrelated files.	Scope boundaries.
Weak validation	Bot claims success without running tests.	scripts/validate.sh.
Unsafe logging	Bot logs tokens or personal data.	Logging rules and review checklist.
Broken deployment	Bot changes config without approval.	Human approval gate.

Risk	Example	Control
Misleading confidence	Bot says code is secure without evidence.	No-guarantee language and test evidence.

Optional Secret Check Script

```
#!/usr/bin/env bash
set -euo pipefail

echo "Checking for secret-like values..."

PATTERN="(AKIA[0-9A-Z]{16})-----BEGIN PRIVATE KEY-----|xox[baprs]-|ghp_[A-Za-z0-9_]{30,}|sk-[A-Za-z0-9]{20,})"

if git grep -n -I -E "$PATTERN" -- . \
  '!package-lock.json' \
  '!pnpm-lock.yaml' \
  '!yarn.lock'; then
  echo "Potential secret found."
  exit 1
fi

echo "No obvious secrets found."
```

12 Rollout Plan

Phase	Add	Goal
Phase 1: Basic Setup	AGENTS.md, .bot/PROJECT.md, .bot/CODING_STANDARDS.md, .bot/DONE_DEFINITION.md, scripts/validate.sh	Give bots basic project context and one validation command.
Phase 2: Security and Review	.bot/SECURITY.md, .bot/REVIEW_CHECKLIST.md, scripts/security-check.sh, CI workflow	Reduce unsafe and unreviewed changes.
Phase 3: Better Tasking	.bot/TASK_TEMPLATE.md, examples/good-task.md, examples/bad-task.md, .bot/current-task.md	Improve human prompts and reduce vague bot output.
Phase 4: Controlled Automation	PR templates, bot labels, CODEOWNERS, required CI checks, dependency review, protected branch rules	Allow bots to help more while preserving human control.

13 Assumptions, Limitations, and Dependencies

Assumptions

- The repository uses Git or another version-control system.
- The team can review bot-generated changes before merge.
- The project has or can add validation commands.
- The organization can adapt the scripts to its language, framework, and CI system.
- The bot has permission to inspect and edit only approved files.

Limitations

- This guide does not guarantee secure, correct, or production-ready code.
- Validation scripts only check what they are designed to check.
- Security conditions may change after publication.
- Bot behavior varies by tool, model, configuration, and repository quality.
- Some projects require additional controls for regulated data, safety-critical systems, financial systems, healthcare systems, or production infrastructure.
- Human review remains required before merge or deployment.

Dependencies

- Repository access
- Development environment
- Package manager
- Test framework
- CI system
- Security review process
- Owner approval process

14 Public Release Checklist and References

Check	Status	Notes
Public-facing template applied	Complete in v1.1	Aligned to GreyNOC Public Document Template GN-TPL-PUB-001 v1.0.
Technical review	Complete for v1.1 QA/QC	Scripts, examples, and operator workflow language checked for public-facing accuracy and readability.
Security review	Complete for v1.1 QA/QC	No credentials, customer-specific data, live environments, or exploit instructions included.
Legal review	Ready for publication record	No guarantees, compliance claims, or liability-sensitive promises added; approval record should be retained by document owner.
Communications review	Complete for v1.1 QA/QC	Tone, release language, contact information, and template alignment checked.
References archived	Complete for release package	Source, final PDF, template reference, and QA/QC notes should be archived with the document record.
Final PDF QA	Complete	PDF rendered and checked for clipping, table readability, headers, footers, spacing, and page-edge bleed.

Source	Use in This Guide	Release Check
GreyNOC Public Document Template GN-TPL-PUB-001 v1.0	Document control, release review, public checklist, assumptions, approval language, and publication metadata.	Internal template reference.
GreyNOC SignalTrace Exposure Reduction Guide v0.1	Current public-review layout pattern, public release rule, content ordering, and publication metadata style.	Internal GreyNOC public-review example.
GitHub Docs	Public context for repository instructions and CI examples.	Verify link before release.
OpenAI Codex Docs	Public context for agent instruction files.	Verify link before release.
Claude Code Docs	Public context for CLAUDE.md memory files.	Verify link before release.
Aider Docs	Public context for repo maps, Git workflow, and scripting patterns.	Verify link before release.

15 Publication Metadata

Field	Status
Website	GreyNOC approved public website.
Contact	GreyNOC approved public inbox, operations contact, legal contact, or media contact.
Copyright	2026 GreyNOC. All rights reserved.
Distribution	Public / Approved external release.
Classification	PUBLIC / APPROVED FOR EXTERNAL RELEASE
Final Release Decision	v1.1 is approved for public release. Publish externally only after GreyNOC owner, technical, security, legal, communications, source archive, final PDF QA, and publication-channel approval are complete.

Final archival requirement

Store the source file, final PDF, approval notes, and checksum in the document record. Public documents should also retain the publication channel and takedown owner.

A Operator Knowledge Expansion

This appendix extends the guide for operators who need to run coding bots in real projects, not just set up the files. It focuses on judgment, scoping, review behavior, failure recovery, and operational discipline.

Operator rule	A coding bot is not the owner of the change. The operator owns the task, the scope, the evidence, and the decision to merge or reject.
----------------------	---

A.1 Operator Mental Model

Effective operators manage coding bots the way a lead engineer manages a fast junior developer: give context, constrain the task, inspect the diff, require proof, and never let speed bypass review.

Operator question	Why it matters	Good answer
What is the exact outcome?	Prevents vague implementation.	The task has measurable acceptance criteria.
What can the bot touch?	Prevents broad rewrites.	Allowed and restricted files are listed.
What must remain unchanged?	Protects adjacent behavior.	Non-goals are explicit.
How will we prove success?	Stops unsupported claims.	Validation commands and expected results are defined.
What is the risk class?	Sets review depth.	Low, medium, high, or blocked until human approval.

A.2 Context Stack and Instruction Precedence

Bots perform better when instructions are layered. Operators should know which instruction source is supposed to win when there is conflict.

Layer	Examples	Operator guidance
Human task	Current issue, ticket, acceptance criteria	Most specific source for what to do now.
Repository instructions	AGENTS.md, CLAUDE.md, Copilot instructions	Persistent rules for how to work in this repo.
Bot docs	PROJECT.md, SECURITY.md, DONE_DEFINITION.md	Project context, security rules, and completion criteria.
Existing code	Patterns, tests, architecture, naming	The bot should preserve the current design unless told otherwise.
Validation output	Test failures, lint errors, CI logs	Evidence overrides confidence. Failed checks must be fixed or explained.
External docs	Framework or vendor docs	Use for current syntax and behavior, especially after tool or library updates.
Conflict handling		When task instructions conflict with security rules, security rules win. When docs conflict with working tests or production behavior, stop and ask for review rather than guessing.

A.3 Bot Capability Levels

Not every bot should have the same authority. Operators should match bot permissions to the risk of the work.

Level	Bot can do	Good for	Human control
L0 Explain	Read snippets and explain.	Learning, debugging, planning.	No file edits.
L1 Draft	Suggest code or patches.	Small functions, tests, docs.	Human applies changes.
L2 Edit	Modify approved files locally.	Scoped bug fixes and	Human reviews diff before

Level	Bot can do	Good for	Human control
		features.	commit.
L3 Validate	Edit and run commands.	Feature work with tests.	Validation logs required.
L4 PR Agent	Open pull requests.	Routine maintenance and low-risk changes.	Protected branches and required review.
L5 Automation	Runs scheduled or event-driven tasks.	Dependency updates and repetitive hygiene.	Strict allowlists, CI gates, and rollback plan.

A.4 Task Decomposition Method

Large requests should be broken into small bot-sized tasks. A good operator decomposes by behavior, risk, and reviewability.

1. Write the desired end state in one sentence.
2. List the smallest independently testable behavior.
3. Separate code changes from documentation, migration, and deployment changes.
4. Identify files the bot may edit and files the bot must not edit.
5. Define acceptance criteria before implementation begins.
6. Require validation evidence after each task, not only at the end of a large project.
7. Merge or reject each small change before expanding scope.

Request type	Bad task shape	Better decomposition
Build a dashboard	One broad bot task.	Data model, API endpoint, UI shell, chart component, tests, docs.
Fix authentication	Unbounded auth rewrite.	Reproduce bug, add failing test, patch one auth path, verify regression.
Clean codebase	Large refactor.	One module, one smell, one measurable outcome.
Improve security	Vague hardening.	Secret scan, input validation, dependency audit, logging review as separate tasks.

A.5 Context Packet for Operators

Before asking the bot to code, prepare a context packet. This reduces guessing and prevents the bot from reading the entire repository without a plan.

Context item	What to include	Example
Problem statement	What is wrong or missing.	Registration accepts malformed email.
Expected behavior	What should happen instead.	Invalid email blocks submission with a clear error.
Relevant files	Likely code and test locations.	src/features/register, src/lib/validation.
Non-goals	What should not change.	Do not change login, password reset, schema, or email provider.
Evidence	Error logs, failing test, screenshot, ticket.	Paste the exact stack trace or test failure.
Validation	Commands and expected outcome.	scripts/validate.sh must pass.

Operator context packet skeleton

Problem:

Expected behavior:

Relevant files:

Do not change:

Acceptance criteria:

Validation commands:

Security notes:

A.6 Validation Strategy Ladder

Operators should choose validation depth based on risk. A documentation edit does not need the same testing as an authentication change, but every change needs some proof.

Risk	Examples	Minimum validation
Low	Docs, comments, formatting.	Spell check, link check, render or preview.
Medium	UI changes, non-critical bug fix.	Format, lint, unit tests, targeted manual review.
High	Auth, payments, data handling, security controls.	Full validation, regression tests, security review, owner approval.
Critical	Production infra, migrations, secrets, access control model.	Change plan, rollback plan, peer review, staged deployment, audit trail.
Evidence rule		The final bot message should not say “should work.” It should say what command was run, what passed, what failed, and what still needs human review.

A.7 Security Triage for Bot-Generated Code

Operators should review every bot-generated diff through a security lens before merge.

Review area	Questions to ask	Red flag
Secrets	Did the bot add tokens, keys, sample credentials, or .env content?	Credential-looking strings in code or docs.
Authentication	Did the bot change login, sessions, cookies, MFA, or token validation?	Checks removed to make tests pass.
Authorization	Did the bot change who can access data or actions?	Role checks simplified or bypassed.
Input handling	Are inputs validated and bounded?	Raw user input reaches shell, SQL, file paths, or templates.
Logging	Does the code log sensitive data?	Tokens, emails, PII, request bodies, or secrets in logs.
Dependencies	Did the bot add packages?	Dependency added without justification or security review.
Error handling	Are errors safe and useful?	Internal stack traces exposed to users.

A.8 Common Bot Failure Modes and Operator Corrections

Failure mode	How it appears	Operator correction
Scope creep	Bot changes unrelated files.	Stop, revert unrelated edits, restate allowed paths.
Test theater	Bot adds shallow tests that only match implementation.	Ask for behavior-based tests and failure cases.
Silent security weakening	Bot removes validation or auth checks.	Reject change and require security-preserving fix.
Dependency sprawl	Bot adds a package for simple logic.	Require standard library or existing dependency first.
Over-refactor	Bot rewrites module while fixing small bug.	Ask for minimal patch and focused diff.
Hallucinated API	Bot uses nonexistent methods or stale syntax.	Require current docs or local compile/test proof.
Validation avoidance	Bot explains why it did not run tests.	Treat as incomplete unless environment truly prevents validation.
Formatting noise	Large unrelated whitespace changes.	Revert noise and preserve review clarity.

A.9 Bot Change Incident Response

If bot-generated code causes a problem, handle it like any other engineering incident. Do not hide the fact that AI assisted the change; document the workflow gap that allowed the issue through.

- Stop the spread: pause related bot tasks and block further automation on the affected repo.
- Contain the change: revert, disable feature flag, roll back deployment, or isolate the affected component.

10. Collect evidence: PR, prompt/task file, changed files, validation logs, CI output, review comments, and deployment timeline.
11. Classify the failure: bad task, bad context, missing test, weak security review, bot ignored instruction, or human approval miss.
12. Fix the product issue with normal engineering controls.
13. Fix the workflow issue by updating AGENTS.md, task templates, tests, scripts, CI, or approval rules.
14. Record lessons learned and verify the same bot task no longer reproduces the failure.

A.10 Operator Command Kit

These commands help operators keep bot-assisted work controlled and reviewable. Adapt them to each project.

```
# Start clean

git status

git checkout -b bot/task-short-name


# Establish baseline

scripts/setup.sh

scripts/validate.sh


# Review bot changes

git status

git diff --stat

git diff


# Validate after changes

scripts/format.sh

scripts/lint.sh

scripts/test.sh

scripts/security-check.sh

scripts/build.sh

scripts/validate.sh


# Review only changed files

git diff --name-only


# Back out noisy or unsafe changes
```

GREYNOC CYBERSECURITY OPERATIONS

Coding Bot Setup Guide
PUBLIC | GN-AI-CODEBOT-001

```
git restore path/to/file
```

```
# Create a reviewable commit only after validation
```

```
git add <approved-files>
```

```
git commit -m "Scoped bot-assisted change"
```

A.11 Model and Agent Selection Guidance

Operators should choose the simplest tool that can safely complete the task. More autonomy is not automatically better.

Need	Preferred mode	Reason
Explain code or error	Assistant / read-only mode	No file-write risk.
Small patch	Draft or limited edit mode	Human can inspect a focused change.
Feature with tests	Agent mode with validation	Bot needs repo access and command output.
Security-sensitive change	Human-led with bot assistance	Operator controls design and review.
Routine maintenance	Guarded PR automation	Useful when CI, branch protection, and rollback are strong.
Production change	Manual approval flow	Bot may assist, but should not own deployment.

A.12 Metrics for Operating Coding Bots

Track whether bots are improving delivery or creating hidden review burden.

Metric	Meaning	Operator action
First-pass validation rate	How often bot changes pass checks.	Improve task template or repo instructions when low.
Scope violation count	How often bots touch unrelated files.	Tighten allowed-path instructions.
Human rework rate	How much code humans rewrite.	Break tasks smaller or add examples.
Security finding count	Security issues introduced by bot changes.	Strengthen SECURITY.md and review checklist.
Dependency additions	Packages added by bots.	Require justification and approval.
Rollback count	Merged bot changes reverted.	Improve validation and release controls.
Time to review	Human review effort.	Reduce diff size and improve summaries.

A.13 Advanced Prompt Patterns

Operators can use prompt patterns to improve reliability. The goal is not to write a longer prompt; the goal is to remove ambiguity.

Pattern	Use when	Prompt language
Plan first	Task has multiple possible approaches.	Before editing, summarize the plan and list files you expect to change.
Minimal diff	Risk of over-refactor.	Make the smallest complete change. Do not reformat unrelated files.
Evidence required	Bot may claim success.	Run scripts/validate.sh and report exact results.
Security hold	Task touches sensitive code.	If the change weakens auth, validation, logging, or permissions, stop and explain.
Test-first	Bug fix or regression.	Add or identify a failing test before changing implementation.
No dependency	Simple utility or validation.	Do not add a dependency unless you explain why existing tools cannot solve it.

A.14 Operator Readiness Checklist

Readiness item	Ready when
Repository instructions	AGENTS.md and .bot files exist and are current.

Readiness item	Ready when
Validation command	scripts/validate.sh runs locally and in CI.
Task quality	Tasks include scope, non-goals, acceptance criteria, and security notes.
Review process	A human reviews every bot-generated diff before merge.
Security controls	Secrets, auth, dependencies, logs, and input handling are checked.
Rollback path	Unsafe bot changes can be reverted quickly.
Metrics	The team tracks validation, rework, scope creep, and security findings.